LI.FI

M0Facet.sol(v1.0.0), IM0OrderBook.sol (v1.0.0)

Independent Security Review

SECURITY RESEARCHER

Sujith Somraaj (somraajsujith@gmail.com)

REPORT PREPARED BY

Sujith Somraaj

Table of Contents

1 About Researcher **1**

2 Disclaimer **1**

3 Scope **1**

4 Risk classification **1**
- 4.1 Impact . . . 1
- 4.2 Likelihood . . . 2
- 4.3 Action required for severity levels . . . 2

5 Executive Summary **2**

6 Findings **3**
- 6.1 Low Risk . . . 3
  - 6.1.1 Solana receiver requirements are not clearly documented . . . 3
  - 6.1.2 Fill deadlines have no upper limit . . . 3
  - 6.1.3 Clear-signing omits the output token, amount, and refund owner . . . 4
- 6.2 Informational . . . 4
  - 6.2.1 Solana output limits and rounding behavior are not documented . . . 4
  - 6.2.2 The OrderBook retains a maximum allowance after an order opens . . . 5
  - 6.2.3 HyperCore is incorrectly treated as an EVM destination . . . 5
  - 6.2.4 M0 orders rely on backend route and token validation . . . 6

1 About Researcher

Sujith Somraaj is a distinguished security researcher and protocol engineer with over eight years of comprehensive experience in the web3 ecosystem.

In addition to working as a Lead Security Researcher at Spearbit, Sujith is also the security researcher and advisor for leading bridge protocol LI.FI and also is a former founding engineer and current security advisor at Superform, a yield aggregator with over $170M in TVL.

Sujith has experience working with protocols / funds including Coinbase, Solana Foundation, Layerzero, Edge Capital, Valinor, Kuru, MNX, Berachain, Optimism, Ondo, Sonic, Monad, Blast, ZkSync, Decent, Drips, SuperSushi Samurai, DistrictOne, Omni-X, Centrifuge, Superform-V2, Tea.xyz, Paintswap, Bitcorn, Sweep n' Flip, Byzantine Finance, Variational Finance, Satsbridge, Rova, Horizen, Earthfast, Angles and multiple other protocols. He has uncovered live bugs in Ethereum, Ripple, Uniswap, Morpho, Injective, Phantom, Cosmos and received bug-bounties from those projects.

Learn more about Sujith on sujithsomraaj.xyz or on cantina.xyz

2 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release, and does not give any warranties on finding all possible security issues of that given smart contract(s) or blockchain software. i.e., the evaluation result does not guarantee against a hack (or) the non existence of any further findings of security issues. As one audit-based assessment cannot be considered comprehensive, I always recommend proceeding with several audits and a public bug bounty program to ensure the security of smart contract(s). Lastly, the security audit is not an investment advice.

This review is done independently by the reviewer and is not entitled to any of the security agencies the researcher worked / may work with.

3 Scope

M0Facet	`src/Facets/M0Facet.sol (v1.0.0)`
IM0OrderBook	`src/Interfaces/IM0OrderBook.sol(v1.0.0)`

4 Risk classification

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

4.1 Impact

Rating	Definition
High	leads to a loss of a significant portion (>10%) of assets in the protocol, or significant harm to a majority of users.
Medium	global losses <10% or losses to only a subset of users, but still unacceptable.
Low	losses will be annoying but bearable — applies to things like griefing attacks that can be easily repaired or even gas inefficiencies.

4.2 Likelihood

Rating	Definition
High	almost certain to happen, easy to perform, or not easy but highly incentivized
Medium	only conditionally possible or incentivized, but still relatively likely
Low	requires stars to align, or little-to-no incentive

4.3 Action required for severity levels

Severity	Required action
Critical	Must fix as soon as possible (if already deployed)
High	Must fix (before deployment if not already deployed)
Medium	Should fix
Low	Could fix

5 Executive Summary

Over the course of 13 hours in total, LI.FI engaged with the researcher to audit the contracts described in section 3 of this document ("scope").

This is a differential review, and hence the scope is limited only to the changes introduced in the audit commit.

In this period of time a total of **7 issues** were found.

Project Summary

Project Name	**M0Facet v1.0.0**
Project Type	Solidity Smart Contracts
Repository	lifinance/contracts
Commit	787b2517
Audit Timeline	October 9, 2026 - October 13, 2026
Methods	Manual Review
Documentation	High
Test Coverage	High

Issues Found

Critical Risk	**0**
High Risk	**0**
Medium Risk	**0**
Low Risk	**3**
Gas Optimizations	**0**
Informational	**4**
Total Issues	**7**

6 Findings

6.1 Low Risk

6.1.1 Solana receiver requirements are not clearly documented

CONTEXT: fill.rs#L288-L305, M0Facet.sol#L51-L53

DESCRIPTION: M0 expects the Solana `recipient` to be the wallet or authority that owns the ATA receiving `tokenOut`. Supplying a token-account address does not direct the transfer to that account. Instead, M0 derives an ATA owned by the supplied address. The solver can create this ATA and fill the order even when the user cannot control the resulting account.

This is easy to get wrong when reusing receiver addresses from another integration. For example, Circle CCTP requires an existing USDC token-account address as its Solana `mintRecipient`. Passing that address as M0's `receiverAddress` creates a second ATA whose authority is the first token account.

If the first token account is itself an ATA, its address is program-derived and cannot sign as the authority of the second account. The delivered tokens therefore cannot be recovered through the normal token program flow. Other token-account addresses are also unsafe unless their account-address key has been deliberately retained and can sign, which typical applications do not require or expect.

The EVM facet cannot inspect a Solana account's type or ownership, so the backend must enforce this requirement. The integration specification also needs to make it explicit: the phrase "Solana recipient's 32-byte public key" does not tell an integrator whether to supply a wallet authority or a token account.

RECOMMENDATION: Clarify in the `M0Data.receiverAddress` NatSpec, backend integration specification, and `docs/M0Facet.md` that the Solana receiver must be the wallet or program authority owning the destination ATA. An SPL Token or Token-2022 account address must not be used as the receiver.

LI.FI: Documented in 1e1065a.

RESEARCHER: Verified documentation added.

6.1.2 Fill deadlines have no upper limit

CONTEXT: M0Facet.sol#L225-L236

DESCRIPTION: The OrderBook rejects deadlines in the past but accepts any future `uint32` timestamp. An order with a deadline near `type(uint32).max` can remain open until 2106.

Only the destination recipient can cancel a cross-chain order before its deadline. `orderOwner` owns the escrow and receives the source-chain refund, but that role does not grant an early cancellation right. If the recipient cannot authorize cancellation on the destination chain, the user must wait for expiry to recover an unfilled order.

This can affect exchange deposit addresses and smart wallets that are not deployed on the destination chain or cannot execute M0's cancellation call. With a deadline set decades into the future, the wait becomes impractical. The M0 Adevar audit highlights the same requirement: a smart-contract recipient must be able to cancel a cross-chain order when the deadline is effectively unlimited.

Expiry does not automatically refund the order. Someone still needs to call `cancelOrder` on the destination chain, pay the required Portal fee, and wait for the cancellation report to release the source escrow. The clear-signing display also omits `fillDeadline`, making it harder for the user to spot an excessively long deadline supplied by mistake or deliberately.

RECOMMENDATION: Set a maximum order lifetime based on the quote and expected settlement time, and enforce `fillDeadline <= block.timestamp + MAX_ORDER_LIFETIME` in the facet. This limits how long LI.FI users can be committed to an order; the OrderBook's existing check only ensures that the deadline is in the future. Consider checking that lower bound locally as well so both limits are enforced together.

Test the deadline boundaries for both entrypoints and document that early cancellation requires a recipient able to authorize it on the destination chain. Run a keeper to cancel expired unfilled or partially filled orders, pay the destination-chain Portal fee, and track the cancellation report until `orderOwner` receives the refund.

LI.FI: Acknowledged - fillDeadline comes from the LI.FI backend's M0 quote, the same trusted calldata builder that

already sets amountOut, orderOwner and tokenOut, so we keep the facet's design of leaving protocol bounds to the OrderBook

RESEARCHER: Acknowledged.

6.1.3 Clear-signing omits the output token, amount, and refund owner

CONTEXT: clearSigningProposal.json#L1221-L1262

DESCRIPTION: Both M0 entrypoints omit `tokenOut`, `amountOut`, `orderOwner`, `refundRecipient`, `solver`, and `fillDeadline` from the signing display. These fields define the output token and amount, ownership of the escrowed order, the recipient of source-chain swap refunds, the solver allowed to fill the order, and the deadline for doing so. The user cannot verify these terms from the clear-signing display before approving the transaction.

A malicious or compromised transaction builder could supply unfavorable terms while the display still shows the source amount, destination, and recipient the user expects. The hidden fields could specify an output token with no value, a negligible output amount, or an order owner who receives the cancellation refund instead of the user.

This requires the transaction builder to supply incorrect calldata before signing; the calldata cannot be changed afterward. Clear-signing is meant to let users verify what they are approving, so leaving these terms hidden defeats that check. The issue was discussed in the PR and remains included under the policy applied across LI.FI integrations.

RECOMMENDATION: Show `tokenOut`, `amountOut`, `orderOwner`, `refundRecipient`, `solver`, and `fillDeadline` for both entrypoints. Use destination-chain formatting for the output token and amount where supported, and show the raw values otherwise. Add tests that change each field and confirm that the signing display reflects the change.

If the signing stack cannot show these fields, bind them to a signed quote or intent commitment that the user can independently verify before considering the integration clear-signable.

LI.FI: Acknowledged - our clear-signing descriptors are generated under one policy for every LI.FI bridge facet (amount, destination chain and recipient, plus the bridge-specific recipient field), so showing M0's output terms would be a generator change across all integrations rather than an M0 fix

RESEARCHER: Acknowledged.

6.2 Informational

6.2.1 Solana output limits and rounding behavior are not documented

CONTEXT: M0Facet.sol#L178-L200

DESCRIPTION: M0 orders store `amountOut` as `uint128`, but Solana's `FillParams.amount_out_to_fill` and SPL token transfers use `u64`. An output amount above `type(uint64).max` therefore cannot be delivered in a single fill. Partial fills can deliver it in smaller amounts, but the recipient's ATA balance is also limited to `u64`. The recipient may need to spend tokens between fills to make room for the rest. Neither the facet nor the documentation makes this requirement explicit.

Separately, scaling rounds the output amount down when the proportional result is not a whole number of destination-token base units. The requested output falls short of the quoted rate by less than one base unit, leaving that fraction in the solver's favor. The economic impact is negligible, but the behavior does not strictly meet the documented goal of preserving all positive swap slippage for the user.

RECOMMENDATION: For Solana destinations, require the output amount after scaling to fit in `uint64` in both the backend and the facet, including the direct entrypoint. If LI.FI intends to support larger orders, document and explicitly support the coordination needed for multiple fills. Test `type(uint64).max` and a quote that starts within the limit but exceeds it after positive swap slippage.

LI.FI: Acknowledged - amountOut comes from the M0 quote built by the LI.FI backend, and the u64 bound (~1.8e19 base units) is far beyond any realistic order even after positive-slippage scaling.

RESEARCHER: Acknowledged.

6.2.2 The OrderBook retains a maximum allowance after an order opens

CONTEXT: M0Facet.sol#L210-L219

DESCRIPTION: The allowance remains after the OrderBook transfers the tokens needed for the current order. Although the deployed implementation only transfers the amount passed to `openOrder`, the approved spender is a transparent upgradeable proxy. A malicious or faulty implementation introduced by an upgrade could use that allowance to transfer other balances of the same token from the Diamond, including tokens sent there after the original M0 transaction.

The Diamond is not intended to hold token inventory, so the expected exposure is limited. A compromise of M0's upgrade authority would also already put active M0 escrow at risk. However, the standing allowance extends that risk to funds held by the Diamond that have no connection to an M0 order. The documentation's statement that no resting funds are at risk overlooks accidental transfers and balances left by other execution paths.

RECOMMENDATION: To reduce this exposure, approve only `bridgeData.minAmount` immediately before calling `openOrder`. The current OrderBook transfers that exact amount, which should consume the allowance. Use an approval helper that supports tokens requiring the existing allowance to be reset before it is changed.

If LI.FI keeps maximum approvals as a policy across integrations, document that the proxy can also access unrelated Diamond balances. Monitor OrderBook upgrades and check after `openOrder` that the expected amount of tokens left the Diamond.

LI.FI: Acknowledged - max approval to the bridge spender is the policy across all LI.FI facets.

RESEARCHER: Acknowledged.

6.2.3 HyperCore is incorrectly treated as an EVM destination

CONTEXT: M0Facet.sol#L329-L339, LiFiData.sol#L12-L22

DESCRIPTION: The function translates LI.FI's Solana chain ID and rejects M0's raw Solana ID. All other values that fit in `uint32` follow the EVM path, so HyperCore resolves to `(1337, false)` instead of reverting. This contradicts the NatSpec assumption that every untranslated LI.FI non-EVM chain ID exceeds `uint32`.

M0's published destination list does not include HyperCore. Assuming the deployed OrderBook uses that configuration, `openOrder` rejects the order with `InvalidDestinationChain` before funds enter escrow. Even so, the facet applies EVM receiver and address validation to a non-EVM destination and relies on the OrderBook to catch the mistake. The `1337` case is not covered by a test.

If M0 enables HyperCore or another non-EVM destination with a `uint32` chain ID, this path would require EVM-shaped recipient, token, and solver addresses and omit `BridgeToNonEVMChainBytes32`. An order could then be accepted with an incorrect recipient or terms that cannot be filled. Chain ID width is not a reliable way to distinguish EVM from non-EVM destinations.

RECOMMENDATION: Reject `LIFI_CHAIN_ID_HYPERCORE` in `_resolveDestination`. Add tests for both the direct and swap entrypoints that expect `InvalidCallData` before the OrderBook is called.

For a lasting fix, explicitly map each supported LI.FI destination to its M0 chain ID and whether it is EVM or non-EVM, and reject unknown destinations. Require that mapping and its tests to be updated when a destination is added. Update the NatSpec and integration documentation to remove the assumption about `uint32` width.

LI.FI: Acknowledged - HyperCore accounts are EVM-format addresses and LI.FI passes them as a plain bridgeData.receiver (as PolymerCCTPFacet does), so the EVM path is the intended handling; the OrderBook currently rejects 1337 before escrow, and if M0 enables it we want those orders to go through.

RESEARCHER: Acknowledged.

6.2.4 M0 orders rely on backend route and token validation

CONTEXT: M0Facet.sol#L225-L237

DESCRIPTION: The M0 OrderBook checks that the destination chain is enabled, but it does not check whether a solver supports the requested token pair. An unsupported order can therefore be accepted and the user's source tokens placed in escrow. Recovering those tokens then requires cancellation on the destination chain and a cancellation report back to the source chain.

Some token types can also cause problems after an order opens. M0 documents that downward-rebasing input tokens can reduce the escrow balance below the amount needed to settle a fill or refund a cancellation, potentially locking funds permanently. Yield earned in escrow may be unrecoverable, and pausable tokens can prevent fills or cancellations. Fee-on-transfer output tokens fail the exact-transfer check. Fee-on-transfer input tokens cause the opening transaction to revert atomically, but that protection does not cover the other cases.

The backend was outside the reviewed PR, so this review does not establish whether those checks are missing from the integration. If it already verifies a fresh M0 quote and enforces a suitable token policy, the remaining work is to document those safeguards and confirm that they are consistently applied.

RECOMMENDATION: Require a fresh M0 quote for the requested token pair before returning calldata. Check the source and destination chains, `tokenIn`, `tokenOut`, `amountIn`, `amountOut`, recipient, solver or exclusivity setting, deadline, and OrderBook address against that quote. Treat `NoQuotesAvailable` as a failure rather than constructing a fallback order. A supported destination or an entry in the supported-assets endpoint does not establish support for a particular pair.

Reject downward-rebasing and fee-on-transfer tokens, and define a policy for yield-bearing and pausable tokens. Cover unsupported pairs, stale quotes, mismatched quote fields, and prohibited token types in backend integration tests. Document the rebase, yield, and pause risks in `docs/M0Facet.md`, including what can happen after funds enter escrow.

LI.FI: Acknowledged - route and token selection, including quote freshness and which token types we route, is enforced by the LI.FI backend, which is outside this PR.

RESEARCHER: Acknowledged.